



UNLOX

Building with LangChain

A practical guide to connecting large language models with external data, memory, and computation — from vector stores to chains.



UNLOX

CHAPTER 14.1

Introduction to LangChain

LangChain is a powerful open-source framework designed to help developers build applications powered by large language models. Rather than writing boilerplate code to manage prompts, parse outputs, and connect to data sources, LangChain provides a structured, modular toolkit that handles the heavy lifting.

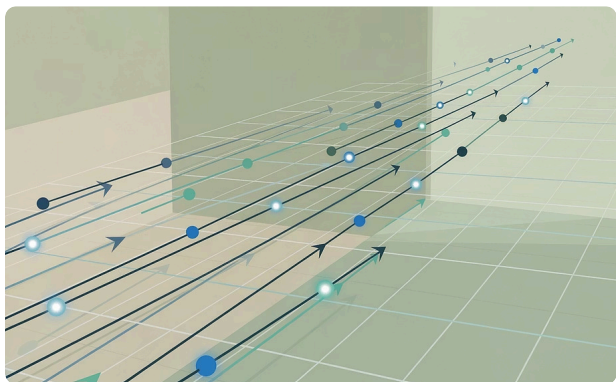
At its core, LangChain simplifies three critical integration challenges: connecting LLMs to **external data sources** for grounded responses, managing **conversational memory** across interactions, and orchestrating **computation** pipelines that go far beyond simple prompt-response cycles. This makes it the go-to framework for production-grade LLM applications.

UNLOCK

KEY COMPONENTS

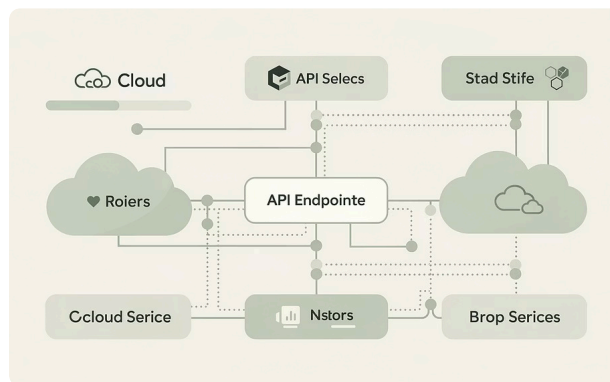
Core Building Blocks

LangChain's architecture is built around composable components that snap together like building blocks. Two foundational pieces you'll work with immediately are **Vector Stores** and the **OpenAI API integration**.



Vector Stores

Specialized databases that store text as numerical vectors — high-dimensional embeddings that capture semantic meaning. This enables **fast similarity searches**, forming the backbone of Retrieval-Augmented Generation (RAG) and other data retrieval techniques. Popular options include Pinecone, Chroma, and FAISS.

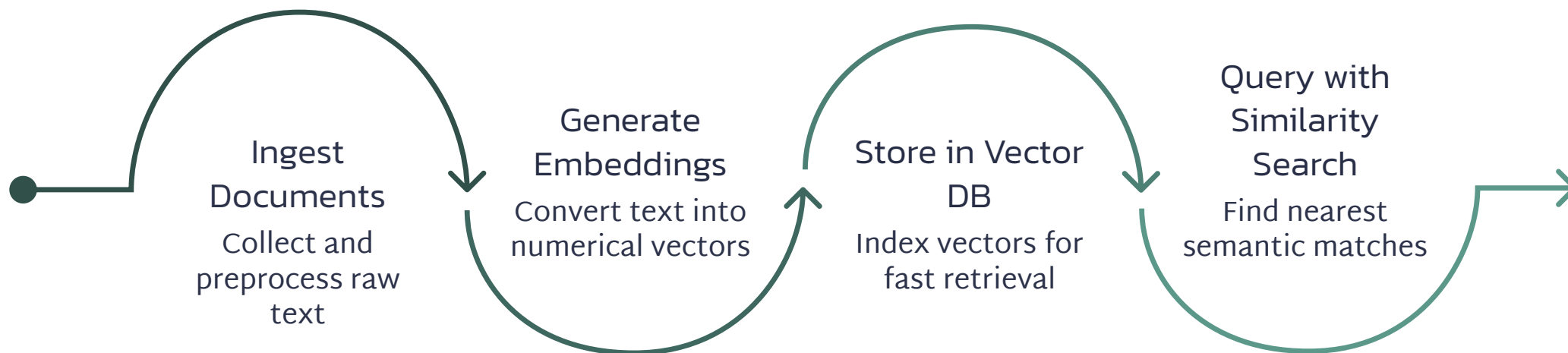


OpenAI API

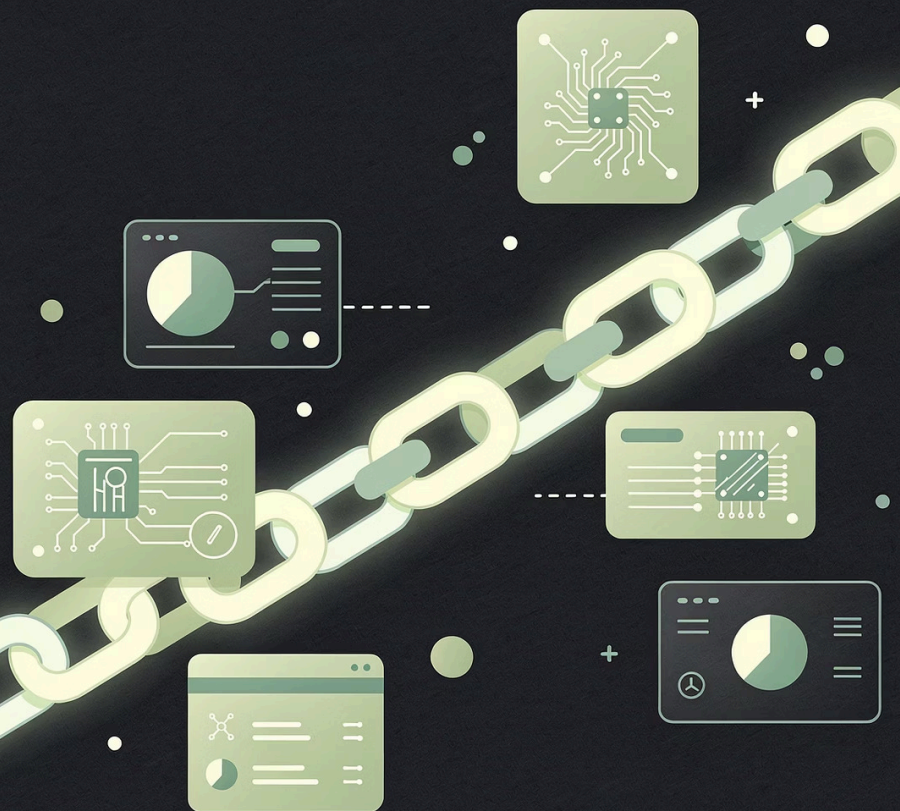
LangChain provides first-class integration with the OpenAI API, giving you access to powerful models like **GPT-4** and **GPT-3.5-turbo**. You'll learn to configure API keys, set model parameters like temperature and max tokens, and seamlessly embed these models into your application workflows.

From Text to Vectors

Understanding how vector stores work under the hood is essential for building effective retrieval systems. The process transforms raw text into searchable, semantically-aware representations.



When a user asks a question, the query is also converted into a vector. The vector store then finds the most semantically similar documents using distance metrics like **cosine similarity**, returning the most relevant context for your LLM to reason over.



UNLOX

CHAPTER 14.2

LangChain Chains

The **Chain** is LangChain's central abstraction — a sequence of components wired together to perform a specific task end-to-end. Think of it as a pipeline where data flows through a series of transformations: a prompt template formats the input, an LLM generates a response, and a parser structures the output. This modular architecture is what makes LangChain so powerful for building production systems.

UNLOCK

ANATOMY OF A CHAIN

Inside a LangChain Chain

Every chain is composed of discrete, reusable components. Understanding each piece lets you mix and match them to build exactly the workflow you need.



Prompt Template

Defines the structure of your input. Variables get injected at runtime, ensuring consistent, well-formatted prompts every time.



LLM Call

The model processes the formatted prompt and generates a raw response. You control parameters like temperature, model choice, and token limits.



Output Parser

Transforms the raw LLM output into structured data — JSON objects, lists, or custom schemas ready for your application logic.

UNLOCK

REAL-WORLD APPLICATIONS

What Can You Build with Chains?

By chaining components together, you unlock complex workflows that would otherwise require significant custom engineering. Here are the most common patterns developers use in production.



Question-Answering Over Documents

Combine a retriever with an LLM to answer questions grounded in your own data — the classic RAG pattern.



Multi-Turn Conversations

Add memory to your chains so the model remembers previous messages and maintains coherent, context-aware dialogue.



Data Extraction

Parse unstructured text into structured formats — extracting entities, dates, and relationships from documents at scale.

UNLOCK

WHY MODULAR?

The Power of Modularity

LangChain's modular approach isn't just elegant — it's **practical**. Each component in a chain can be independently designed, tested, and swapped out without rewriting your entire application. This means you can:

- **Swap models** — switch from GPT-3.5 to GPT-4 (or an open-source alternative) with a single line change
- **Debug in isolation** — test your prompt template separately from parsing logic
- **Reuse components** — share prompt templates and parsers across multiple chains
- **Scale incrementally** — start simple and add complexity as your requirements grow



Pro Tip

Start with a simple **LLMChain** (prompt + model + parser) and validate your logic before adding retrievers, memory, or routing. This iterative approach prevents complexity from becoming overwhelming.

A Typical LangChain Architecture

In a production application, chains don't exist in isolation. They connect to vector stores for retrieval, memory systems for conversation history, and external tools for computation. Below is a typical flow for a **document Q&A chatbot**:

01

User submits a question

The raw query enters the system through your application's interface.

02

Retriever queries the vector store

The question is embedded and matched against stored document vectors to find relevant context.

03

Context + question are injected into a prompt template

The retrieved documents and original question are formatted into a structured prompt.

04

LLM generates a grounded answer

The model reasons over the provided context, producing an accurate, cited response.

05

Output parser structures the response

The raw output is cleaned, formatted, and returned to the user — ready for display.

What You've Learned

1

LangChain Simplifies LLM Development

It provides a structured framework for connecting models to data sources, memory, and external tools — eliminating boilerplate.

2

Vector Stores Enable Semantic Search

By converting text into numerical embeddings, vector stores power fast similarity searches that form the core of RAG-based applications.

3

Chains Are Your Primary Abstraction

Sequences of prompt templates, LLMs, and parsers snap together into reusable, testable workflows for any task.

4

Modularity Is the Superpower

Build iteratively, swap components freely, and scale complexity without rewriting — this is how production LLM apps are built.

